

Instituto Tecnológico de Costa Rica
Escuela de Ingeniería en Computadores
Curso: Introducción a la Programación

Proyecto 1

Juego 2D de Plataformas

Murcian Hunter

Estudiante:	Francisco Li Ruiz
Carné:	2026014113
Grupo:	2
Profesor:	Jeff Schmith
Semestre:	Primer Semestre, 2026
Fecha de entrega:	10/05/2026

1. Objetivo del Proyecto

Murcian Hunter es un videojuego 2D de plataformas desarrollado en Python con la biblioteca Tkinter. El objetivo principal es que el jugador, controlando al personaje Hunter, se desplace desde un punto de inicio hasta una meta final dentro de un escenario compuesto por plataformas y escaleras, esquivando o superando enemigos en el camino.

Para ganar, el jugador debe llegar al portal cian (meta) sin quedarse sin vidas. Los elementos esenciales del nivel son:

Plataformas: Superficies sobre las que Hunter puede caminar y saltar.

Escaleras: Permiten al jugador subir o bajar entre niveles del escenario.

Enemigos: Dos tipos de murciélagos enemigos que representan obstáculos reales.

Murciélagos coleccionables: Objetos que caen y suman puntos al atraparlos.

Meta: Portal cian con borde amarillo que marca la condición de victoria.

Punto de inicio: Posición inicial del personaje claramente definida en el mapa.

2. Planteamiento General de la Solución

Representación del escenario

El escenario se representa mediante listas de Python. Cada elemento del mapa (plataformas, escaleras, enemigos) es una lista con sus coordenadas, dimensiones, ID de canvas y color. El nivel predeterminado se organiza en un diccionario llamado *pantallas*, donde cada llave numérica corresponde a una zona del mapa con sus propias plataformas y escaleras. Esto permite cambiar de zona simplemente actualizando un índice.

Movimiento del jugador

El movimiento se controla mediante un diccionario de teclas (*teclas*) cuyos valores son booleanos. Al presionar una tecla se activa el interruptor correspondiente y al soltarla se apaga. La función *mover_hunter()* lee estos interruptores en cada cuadro y actualiza la posición del personaje. Se incluye también un reseteo de teclas al perder el foco de la ventana para evitar que el personaje quede moviéndose solo.

Sistema de salto y física

El salto funciona asignando una velocidad vertical negativa al personaje (*POTENCIA_JUMP* = -15). En cada cuadro se suma la gravedad (*GRAVEDAD* = 0.8) a esa velocidad, simulando una parábola natural. Un booleano en *hunter[3]* indica si el personaje está en el suelo, condición necesaria para poder saltar.

Escaleras

Se detecta si Hunter está dentro de los límites de una escalera comparando sus coordenadas con las de cada elemento de la lista *escaleras*. Mientras esté en la escalera, la gravedad se suspende y el jugador puede moverse verticalmente con espacio (subir) o flecha abajo (bajar). También puede saltar lateralmente desde la escalera.

Estado del juego en el día 2 de desarrollo

La siguiente imagen muestra el estado del proyecto al segundo día de desarrollo (27 de abril de 2026). En esta etapa ya se contaba con plataformas funcionales, colisiones básicas y el personaje representado como un rectángulo morado, antes de integrar los sprites finales.



Figura 1. Estado del juego al día 2 de desarrollo (Abril 27, 2026).

Detección de la meta

La colisión con la meta se verifica mediante una comparación de rectángulos (AABB) entre las coordenadas de Hunter y las de la meta. Si se produce el contacto y el juego no está en pausa, se llama a *mostrar_pantalla_final()* con el mensaje de victoria.

3. Generación y Validación del Mapa

Nivel predeterminado

El nivel predeterminado fue diseñado manualmente por el programador para garantizar que siempre sea posible alcanzar la meta. El escenario se divide en dos pantallas: la primera contiene plataformas y una escalera central que permite subir; la segunda presenta la meta y escaleras laterales. El diseño

asegura que siempre existe un camino válido desde el inicio hasta la meta.

Editor de mapas

El editor de mapas permite al jugador construir su propio nivel colocando plataformas, escaleras, enemigos, punto de inicio y meta mediante clics en el canvas. Antes de iniciar la partida con el mapa personalizado, el sistema valida que se hayan colocado obligatoriamente un punto de inicio y una meta. Si alguno falta, se muestra un mensaje de error y no se permite iniciar. Esta validación se implementa en la función *probar_mapa_pro()*.

Sistema de puntaje del editor

El puntaje se basa en la recolección de los murciélagos amarillos que caen del cielo, entre más recolecte hunter, más puntos se obtendrán.

4. Funciones Principales

4.1 `mover_hunter()`

Es el núcleo del movimiento del juego. Se ejecuta en cada cuadro de la animación y se encarga de leer el estado de las teclas, aplicar la física del salto y la gravedad, detectar si el personaje está en una escalera, verificar colisiones con plataformas y gestionar el cambio de pantalla. Sin esta función el juego no tendría mecánicas de movimiento funcionales.

4.2 `animove()`

Es el bucle principal del juego. Usando *ventana.after(20, animove)* se llama a sí misma cada 20 ms, creando el ciclo de animación. Además de llamar a *mover_hunter()*, actualiza la imagen de Hunter según su dirección, gestiona los murciélagos coleccionables y los enemigos, y actualiza la posición del sprite en el canvas. Es el equivalente al game loop del juego.

4.3 `cargar_npantalla(num)`

Gestiona el cambio de zona del nivel predeterminado. Elimina del canvas todos los elementos de la pantalla anterior (plataformas, escaleras, enemigos, meta) y dibuja los de la nueva pantalla. Esto permite tener un mapa extendido en múltiples zonas sin necesidad de múltiples ventanas.

4.4 `mostrar_pantalla_final(mensaje, color, puntos_finales)`

Se encarga del flujo de fin de partida. Detiene el juego, carga la música correspondiente (victoria o derrota), muestra la ventana de resultado con el puntaje obtenido y ofrece el botón para volver al menú. También llama a *guardar_puntajes()* para registrar el resultado en el archivo de texto.

4.5 `click_editor(event)`

Maneja toda la lógica del editor de mapas. Según el tipo de elemento seleccionado por el usuario, calcula las coordenadas ajustadas a la cuadrícula de 20x20 píxeles, dibuja el elemento en el canvas del editor y guarda sus datos en las listas correspondientes del mapa personalizado para su uso posterior al jugar.

5. Problemas Encontrados y Solución Aplicada

La meta no aparecía al probar el mapa del editor

La meta se creaba con `state='hidden'` en `abrir_ventana_juego()` y dependía de `cargar_npantalla()` para hacerse visible, función que nunca se llama en el editor por tener una sola pantalla. Se solucionó separando la lógica: en el modo editor la meta se dibuja directamente visible, y en el nivel predeterminado inicia oculta hasta que el jugador llega a la pantalla 2.

El personaje avanzaba solo hacia la derecha

Al cambiar de pantalla o abrir la ventana de ajustes, la ventana perdía el foco momentáneamente y el evento `KeyRelease` no se registraba, dejando el interruptor de la tecla derecha en `True` permanentemente. Se solucionó vinculando el evento a una función que resetea todos los interruptores de teclas.

Error de sintaxis en `meta_personalizada.get()`

Se usaban corchetes en lugar de paréntesis (`meta_personalizada.get[1, [None]]`) lo que causaba un crash al cambiar de pantalla en modo editor. Se corrigió usando la sintaxis correcta con paréntesis.

La función `ir_a_juego()` pisaba el interruptor `es_mapa_personalizado`

La función reseteaba `es_mapa_personalizado[0]` a `False`, anulando la configuración del editor antes de iniciar la partida. Se separó en dos funciones: `ir_a_juego()` para uso general e `ir_a_juego_base()` que establece el interruptor en `False` solo cuando se inicia el nivel predeterminado desde el menú.

Hundimiento del personaje en plataformas

En las primeras versiones el personaje se hundía ligeramente dentro de las plataformas al aterrizar. Se resolvió ajustando la condición de colisión para verificar si los pies de Hunter cruzan el tope de la plataforma y teletransportarlo exactamente encima del bloque al detectar la colisión.

6. Conclusión

El proyecto Murcian Hunter logró implementar un videojuego 2D de plataformas completo y funcional utilizando únicamente Python con Tkinter y Pygame para el audio, respetando la restricción de no usar clases. Se implementaron todas las funcionalidades requeridas: movimiento del personaje, física de salto con gravedad, interacción con escaleras, dos tipos de enemigos con comportamientos distintos, sistema de vidas y puntos, editor de mapas con validación, registro de puntajes en archivo de texto,

música y pantallas de victoria y derrota.

Lo más importante aprendido durante el desarrollo fue la gestión del estado del programa mediante listas y diccionarios mutables en lugar de clases, lo que exigió un diseño cuidadoso de cada estructura de datos. Los aspectos más retadores fueron la sincronización del ciclo de animación con los eventos de teclado, la detección de colisiones precisa y el manejo del flujo entre múltiples ventanas de Tkinter sin que interfirieran entre sí.

7. Referencias

Anthropic. (2026). *Claude* [Modelo de lenguaje de IA]. <https://claude.ai> — Utilizado como apoyo en la depuración de errores, corrección de bugs y consultas de lógica durante el desarrollo del proyecto.

Tkinter Documentation. (s.f.). *tkinter — Python interface to Tcl/Tk*. Python Software Foundation. <https://docs.python.org/3/library/tkinter.html>

Pygame. (s.f.). *pygame.mixer — pygame v2 documentation*. <https://www.pygame.org/docs/ref/mixer.html>

Codewithrandom. (2023). *Diccionarios en Python: uso y manejo avanzado* [Video]. YouTube. <https://youtu.be/vAy4IM7NLIQ?si=F1ToTrnOHORsVI1M>